Cloud-First Autopilot Zero-Touch Provisioning (Entra Join, Optional Later Domain Join) — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 5

The situation . . . 5

The complication . . . 5

The resolution . . . 5

The outcome . . . 6

Bottom line . . . 6

Solution Overview — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 6

What this solution delivers . . . 6

The zero-touch model in GCC High (plain language) . . . 6

Commercial vs GCC High — what's different . . . 7

The standard deployment model (all solutions) . . . 9

Named objects this solution creates (used verbatim throughout) . . . 9

Who this is for . . . 9

Sources . . . 10

Requirements — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 10

R1. A GCC High (or DoD) tenant on the US sovereign cloud . . . 10

R2. US Government licensing that carries Intune + Entra ID P1/P2 . . . 10

R3. Administrative roles in the GCC High tenant . . . 11

R4. Supported Windows 11 build on target devices . . . 11

R5. Microsoft Entra join is the only supported join type . . . 11

R6. The "Intune Provisioning Client" service principal exists and can own a group . . . 11

R7. Users are allowed to join devices to Microsoft Entra ID . . . 12

R8. Automatic (MDM) enrollment is enabled for Windows . . . 12

R9. Sovereign network endpoints allowed through filtering . . . 12

R10. Understand the GCC High servicing model (no Autopatch / update policies) . . . 13

Assess gate (pass/fail) . . . 13

Sources . . . 13

Global Requirements — Intune Solutions (category-level) 14

G1. Microsoft Entra ID tenant . . . 14

G2. Microsoft Intune subscription . . . 14

G3. Administrative roles . . . 14

G4. Device enrollment in Intune . . . 14

G5. Pilot security group . . . 14

G6. Supported Windows baseline . . . 14

G7. Network access to Microsoft cloud endpoints . . . 14
Implementation Guide — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 15
Phase A — Configure (build the objects) . . . 15
Phase B — Pilot (provision real devices) . . . 19
Sources . . . 19
Testing & Validation — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 20
T1 — Configuration objects exist and are correct (pre-pilot) . . . 20
T2 — Tenant is genuinely GCC High (not Commercial) . . . 20
T3 — Endpoint reachability from the provisioning network . . . 21
T4 — End-to-end pilot provisioning (the core test) . . . 21
T5 — Deployment monitoring reflects the run . . . 21
T6 — Servicing and policy land (post-provisioning) . . . 22
Validate gate (pass/fail summary) . . . 22
Sources . . . 22
Troubleshooting — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 22
1. OOBE completes but the device group is empty and nothing installed . . . 22
2. The device group is Dynamic, not Assigned . . . 23
3. "I don't see Windows Autopilot / Deployment Profiles / hardware-hash import" . . . 23
4. Scripts return nothing / see the "wrong" tenant . . . 23
5. Provisioning fails at sign-in / can't reach services . . . 23
6. Device won't provision — unsupported Windows build . . . 23
7. Self-deploying / pre-provisioning / device-rename options are missing or ignored . . . 24
8. Too many apps during OOBE / low provisioning success rate . . . 24
9. Windows Hello for Business PIN reset doesn't work . . . 24
10. Update/servicing policies don't appear or don't apply . . . 24
11. Compliance policy always evaluates non-compliant on an attestation rule . . . 24
12. Certificate provisioning expected via Cloud PKI . . . 24
13. Remote assistance for a stuck device isn't available . . . 24
14. Device was previously a classic Autopilot device elsewhere . . . 25
15. RBAC admin can't configure device preparation . . . 25
Escalation checklist (attach to any support case) . . . 25
Sources . . . 25
Best Practices — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 25
Portals & tooling . . . 25
Provisioning design (the supported path only) . . . 26

Servicing (no Autopatch in GCC High) . . . 26
Compliance & security . . . 26
RBAC & governance . . . 27
Operations & change control . . . 27
Sources . . . 27
Deliverables Summary — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High) 28
Documents (`source/`) . . . 28
PowerShell scripts (`scripts/`) . . . 28
Named objects created (verbatim) . . . 29
Suggested order of use . . . 29
The one thing to remember . . . 30
Appendix A — PowerShell Scripts 31

Executive Summary — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

The situation

Your organization operates on **GCC High** (or DoD) — the US sovereign cloud required for regulated government and defense-industrial-base workloads (CMMC, ITAR, DFARS). You want the same outcome commercial enterprises get from cloud-first provisioning: a new Windows 11 device is powered on, the user signs in, and it configures itself into a managed, compliant, ready-to-work state with **no imaging and no technician touch**.

The complication

GCC High is not the commercial portal with a different sign-in. It is a physically separate cloud with `*.us` admin portals, sovereign service endpoints, and — decisively — **a reduced feature set**. Most consequentially for this workload:

> ***Classic Windows Autopilot is not available in GCC High, and Microsoft has no plan to bring it.*** *Teams that lift-and-shift a commercial Autopilot runbook build against a hardware-hash import and "Deployment Profiles" blade* ***that do not exist here*** *— wasting a project cycle before discovering the gap.*

Additional gaps that quietly break commercial designs: **Windows Autopatch and the newer update policies are unavailable**, **Microsoft Store for Business, Cloud PKI, Device Health Attestation, DFCI, and Remote Help are unavailable**, and self-deploying/pre-provisioning modes and OOBE device-rename are **not yet available**.

The resolution

This solution delivers zero-touch provisioning on the **one supported path**: **Windows Autopilot device preparation** (user-driven, Microsoft Entra join), which **is** supported in GCC High/DoD. It provides:

- The correct `*.us` **portal navigation** end to end, with every commercial-vs-GCC-High

difference called out at the step where it matters.

- A named, reusable object set (provisioning users group, **assigned** device group owned by

the **Intune Provisioning Client** service principal, device preparation profile, enrollment status page, least-privilege RBAC role, update ring, compliance policy).

- **Five PowerShell scripts** that connect to **US Gov Graph** (`-Environment USGov`) for

read-only discovery, validation, and reporting, plus **safe** remediation and scope targeting that default to `-WhatIf` (preview before change).

- A **cited GCC High parity reference** separating doc-confirmed facts from items flagged

"verify for GCC High," so nothing is taken on faith.

- Testing and troubleshooting with **pass/fail gates** and the GCC-High-specific failure

modes (most notably: the empty-device-group symptom when the provisioning service principal is not the group owner).

The outcome

Cloud-first Windows provisioning stood up **correctly on GCC High** — right portals, right supported technology, parity gaps handled, servicing designed around update rings rather than unavailable Autopatch — and **proven on a pilot** before you scale, using the standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model.

Bottom line

Self-guided, tested, sovereign-correct — at a fraction of the cost of a consulting engagement, and without the failed first attempt that comes from running commercial steps on a government tenant.

All feature-availability claims are drawn from Microsoft US Government documentation; see `gcc-high-parity-notes.md` for the full cited list and any "verify for GCC High" flags.

Solution Overview — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

What this solution delivers

This is the **Government (GCC High) edition** of Cloud-First Autopilot Zero-Touch Provisioning. The goal is unchanged from the Commercial edition: a new Windows 11 device ships from the OEM, an end user powers it on, signs in with their organizational account, and — with **no imaging, no manual domain join, and no technician touch** — the device provisions itself into a managed, compliant, ready-to-work state. The device is **Microsoft Entra joined** (cloud-first), with the option to add on-premises resource access later.

What is different here is **where** and **how** you build it. GCC High is a physically separate, US-sovereign cloud with its own admin portals (`*.us`), its own service endpoints, and — critically — **a different feature set** than Commercial. Some capabilities you would reach for in a Commercial Autopilot design **do not exist in GCC High**. Following Commercial steps on a GCC High tenant does not error politely; it silently sends you to the wrong portal or has you look for a blade that was never provisioned. This edition is authored so a government tenant is configured **correctly**.

> ***The single most important fact in this solution:*** *Classic **Windows Autopilot is NOT available in GCC High or DoD** and there is **no plan to bring it** (Microsoft, *Intune Government Service overview*, "Intune features not available in GCC High and DoD"). The supported zero-touch path in GCC High is **Windows Autopilot device preparation** (user-driven, Microsoft Entra join). This solution is built entirely on the supported path.*

The zero-touch model in GCC High (plain language)

1. You publish a **Windows Autopilot device preparation** profile in the GCC High Intune admin center (`https://intune.microsoft.us`).

1. The profile points at an **assigned** Microsoft Entra security group (the *device

group*) whose **owner** is the **Intune Provisioning Client** service principal. This ownership is the mechanism (called *Enrollment Time Grouping*) that lets Intune drop each freshly enrolled device into the group **at enrollment time**, so apps, scripts, and policies deploy immediately.

1. A user in your **provisioning user group** is allowed to Entra-join devices and is

licensed for Intune.

1. The user powers on a supported Windows 11 device, connects to network, signs in with

their GCC High account (`...@yourtenant.onmicrosoft.us` or your routed domain), and the device Entra-joins, MDM-enrolls, installs the selected apps/scripts, applies policy, and lands the user on the desktop with a progress percentage the whole way.

No serial-number hardware hash upload is required for the basic user-driven flow (that is a *classic Autopilot* concept, which does not apply here). Optionally, **corporate identifiers** can be pre-uploaded if you use enrollment restrictions to block personal devices.

Commercial vs GCC High — what's different

This is the section to read before you touch anything. Every difference below is doc-confirmed against Microsoft US Gov documentation unless explicitly flagged *"verify for GCC High."*

1. The provisioning technology itself is different

Capability	Commercial	GCC High / DoD
Classic Windows Autopilot (deployment profiles, hardware-hash registration, deployment groups)	Supported	**Not available. No plan to support.** (*Intune Government Service overview*)
Windows Autopilot device preparation (user-driven, Entra join)	Supported	**Supported** (*Intune Government Service overview*; *Autopilot device preparation overview*)
Self-deploying mode / pre-provisioning (white-glove)	Supported (classic Autopilot)	**Not yet supported** — "planned" (*Intune Government Service overview*)
Rename device during OOBE based on org structure	Supported	**Not yet supported** — "planned"
DFCI management (UEFI/BIOS lock via Autopilot)	Supported	**Not available**

Consequence for design: you cannot design around a hardware-hash CSV, a "Deployment Profiles" blade, a "Windows Autopilot devices" import, self-deploying kiosks, or Autopilot device renaming. This solution uses **device preparation profiles** and **Entra Join + enrollment-time grouping** instead. Where a Commercial runbook says "Autopilot," read "Autopilot **device preparation**."

2. The portals are different (`*.us`, not `*.com`)

Purpose	Commercial URL	GCC High URL
Intune admin center	`intune.microsoft.com`	`https://intune.microsoft.us`
Entra admin center	`entra.microsoft.com`	`https://entra.microsoft.us` *(verify for GCC High — see parity notes)*
Azure portal	`portal.azure.com`	`https://portal.azure.us`
Microsoft 365 admin	`admin.microsoft.com`	`https://portal.office365.us`
Sign-in / identity (Evo STS)	`login.microsoftonline.com`	`https://login.microsoftonline.us`
Company Portal service	`portal.manage.microsoft.com`	`https://portal.manage.microsoft.us`
Microsoft Graph	`graph.microsoft.com`	`https://graph.microsoft.us`

(Portal and endpoint URLs: Microsoft, *Network endpoints for US government deployments*.)

3. Update and servicing capabilities are reduced

Several servicing features a Commercial design leans on **do not exist in GCC High**:

- **Windows Autopatch** — not available.
- **Feature updates, Quality updates, Expedite updates, Driver updates for Windows in Intune** — not available.
- **Windows Subscription Activation** (auto step-up Pro → Enterprise) — not available.
- **Delivery Optimization for Win32 apps** — not available.

Consequence: in GCC High you service Windows through **Update rings for Windows 10 and later** (which *are* available) rather than the newer per-update-class policies or Autopatch. Do not build a rollout that assumes Autopatch or feature-update policies.

4. Other feature gaps that touch provisioning

- **Microsoft Store for Business** — not available (do not plan Store-for-Business app provisioning; use Win32/line-of-business apps and the newer Store integration where supported).
- **Remote Help** — not available (plan an alternative remote-assist tool for post-provisioning support).
- **Cloud PKI** — not available in GCC High.
- **Device Health Attestation** — not available.
- **Windows Hello for Business** — the WHfB policy path is available, but the **Microsoft PIN reset service is not available in Azure Government**; PIN recovery must be configured differently. *(See parity notes.)*

5. What is the same

The core of the deployment is genuinely the same experience: **Entra Join**, **automatic MDM enrollment**, **assigned device group with the Intune Provisioning Client as owner**, **app and PowerShell-script delivery during OOBE**, **configuration and compliance policies**, and **near-real-time deployment**

monitoring. The muscle memory transfers; the portals and the servicing model do not.

The standard deployment model (all solutions)

This solution follows the standard, gated model. **Do not skip a gate.**

Assess → Configure → Pilot → Validate → Expand.

- **Assess** — confirm every prerequisite in `02-requirements.md` (GCC High tenant, US Gov licensing, roles, sovereign portal access, supported Windows 11 build, endpoints allowed through filtering). *Pass/fail gate:* every requirement verifies before you configure.
- **Configure** — build the named objects in `03-implementation.md` using the GCC High portals.
- **Pilot** — provision a small set of real devices for the pilot group.
- **Validate** — run the tests in `05-testing.md`; every gate must pass.
- **Expand** — widen scope group by group using `CFAPG-Set-ScopeTargeting.ps1` (default `-WhatIf`).

Named objects this solution creates (used verbatim throughout)

Reuse these exact names in every step and script so the pieces line up:

Object	Exact name
Provisioning user group (allowed to Entra-join, Intune-licensed)	`CFAPG-Provisioning-Users`
Device security group (assigned; owner = Intune Provisioning Client)	`CFAPG-Autopilot-Devices`
Pilot device/user group	`CFAPG-Pilot`
Autopilot device preparation profile	`CFAPG-DevicePrep-UserDriven-EntraJoin`
Enrollment Status Page profile	`CFAPG-ESP-Provisioning`
Custom Intune RBAC role	`CFAPG Device Preparation Administrator`
Windows update ring	`CFAPG-UpdateRing-Pilot`
Compliance policy	`CFAPG-Compliance-Win11`
Deployment scope tag	`CFAPG`

Who this is for

Novice-to-intermediate administrators standing up cloud-first Windows provisioning on a **GCC High** (or DoD) tenant who need the correct sovereign portals, the supported provisioning technology, and the Commercial-vs-GCC-High gaps called out — instead of a Commercial runbook that silently fails.

Sources

• Microsoft, *Microsoft Intune Government Service overview* (GCC High / DoD feature availability, planned, and not-available tables).

• Microsoft, *Overview of Windows Autopilot device preparation* (GCCH/DoD support, Entra-join-only, Enrollment Time Grouping).

• Microsoft, *Windows Autopilot device preparation requirements* (software, networking, licensing, configuration, RBAC).

• Microsoft, *Network endpoints for US government deployments* (portal and service URLs).

Requirements — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

Confirm **every** requirement below before you configure anything (the **Assess** gate). Category-level prerequisites common to all Intune solutions live in `global-requirements-intune-solutions.md`; the items here are the ones **specific to this solution and to GCC High**. Each entry follows: **Description · Why · Verify · Implement.**

> ***GCC High portal reminder.*** *Everywhere the global requirements say* `entra.microsoft.com` / `intune.microsoft.com`*, on GCC High you use* `https://entra.microsoft.us` **(verify for GCC High)* and* `https://intune.microsoft.us` *and sign in at* `https://login.microsoftonline.us`*.*

R1. A GCC High (or DoD) tenant on the US sovereign cloud

Description: An active Microsoft 365 / Microsoft Entra tenant provisioned in the **Government Community Cloud High (GCC High)** or **DoD** environment (built on Azure Government / IL4–IL5), **not** a Commercial or GCC tenant.

> ***Note (doc-confirmed):*** *For Intune,* ***GCC is the *commercial* instance****; only* ***GCC High*** *and* ***DoD*** *are the separate government cloud. If your tenant is "GCC," you are on the Commercial Intune service and should use the Commercial edition of this solution. (Microsoft, *Intune Government Service overview*.)*

Why: Autopilot device preparation and the `*.us` portals only exist on the government cloud; a Commercial tenant will not expose them and cannot be migrated in place (devices must unenroll/re-enroll to move clouds). **Verify:** Sign in to `https://intune.microsoft.us`. It loads and shows your tenant. Signing in at `intune.microsoft.com` with the same account should **not** show your government tenant's Intune data. **Implement:** Purchase the appropriate GCC High / DoD licensing through an authorized government-cloud channel and complete tenant onboarding. There is no in-place move from Commercial.

R2. US Government licensing that carries Intune + Entra ID P1/P2

Description: A US Gov subscription that includes Intune and Microsoft Entra ID P1 or P2, e.g. **Microsoft 365 G3 / G5 (Government)**, **EMS E3/E5 (Gov)**, or **Entra ID P1/P2 + Intune Plan 1 (Gov)**. **Why:** Autopilot

device preparation needs automatic MDM enrollment and company-branding capabilities (Entra ID) plus Intune as the MDM. (Microsoft, *Autopilot device preparation requirements → Licensing*.) **Verify:** `https://portal.office365.us` → **Billing** → **Licenses**; confirm a Gov SKU that lists Intune and Entra ID P1/P2. **Implement:** Assign a qualifying Gov license **to each provisioning user** (a Microsoft 365 subscription still requires per-user license assignment so the user can enroll the device in Intune).

R3. Administrative roles in the GCC High tenant

Description: The operator holds **Intune Administrator** (and, for the one-time tenant setup steps, an account with **Global Administrator**). **Why:** Creating the device preparation profile, enrollment settings, ESP, and the RBAC role requires elevated Intune rights; enabling automatic enrollment and consenting the provisioning service principal are tenant-wide. **Verify:** `https://entra.microsoft.us` *(verify for GCC High)* → **Roles and administrators** → confirm **Intune Administrator** on the operator account. **Implement:** Assign **Intune Administrator**; reserve **Global Administrator** for the tenant-wide steps only. Later, hand day-to-day work to the least-privilege custom role `CFAPG Device Preparation Administrator` (built in `03-implementation.md`).

R4. Supported Windows 11 build on target devices

Description: Target devices run **Windows 11, version 24H2 or later**, **or** Windows 11 22H2/23H2 **with KB5035942 or later** included in the OEM/installation media. Supported editions: Pro, Pro Education, Pro for Workstations, Enterprise, Education, Enterprise LTSC. **Why:** Autopilot device preparation depends on client features present only in these builds. Windows 10 is **not** supported for device preparation. (Microsoft, *Autopilot device preparation requirements → Software*.) **Verify:** On a sample device run `winver`; confirm Windows 11 with a build that includes KB5035942 (April 2024 media or later), or 24H2+. **Implement:** Order OEM devices with 24H2 (or 22H2/23H2 media dated April 2024+). Do not attempt device preparation on Windows 10.

R5. Microsoft Entra join is the only supported join type

Description: Devices provision as **Microsoft Entra joined** (cloud-first). Hybrid Entra join is **not** supported by Autopilot device preparation. **Why:** Device preparation supports **Entra join only**; this is also why the "optional later domain join" in the title means *adding on-prem resource access after the fact* (e.g. via Entra hybrid access / cloud Kerberos trust), **not** provisioning as hybrid-joined. (Microsoft, *Autopilot device preparation overview → Requirements*.) **Verify:** `https://entra.microsoft.us` *(verify for GCC High)* → **Devices** → **Device settings** → confirm users are allowed to Entra-join devices (set precisely in R7). **Implement:** Plan a cloud-first Entra-join deployment. If on-prem resource access is needed, plan it as a **post-provisioning** add-on (out of scope for the zero-touch provisioning itself).

R6. The "Intune Provisioning Client" service principal exists and can own a group

Description: The service principal **Intune Provisioning Client** — AppId `f1346770-5b25-470b-88bd-d5744ab7952c` — is present in the tenant (it may appear as **Intune Autopilot ConfidentialClient**; match by **AppId**). **Why:** Autopilot device preparation uses **Enrollment Time Grouping** — Intune adds each newly enrolled device to the assigned device group **only** if this service principal is an **owner** of that group. Without the owner, the group stays empty and **nothing**

deploys during OOBE. (Microsoft, *Create an assigned device group*; device preparation troubleshooting.) **Verify:** `https://entra.microsoft.us` *(verify for GCC High)* → **Enterprise applications** → search the AppId `f1346770-5b25-470b-88bd-d5744ab7952c`. **Implement:** If missing, it is created when you first configure Autopilot device preparation / automatic enrollment; open the Intune admin center once as Global Admin to trigger provisioning, then re-check. `CFAPG-Validation.ps1` checks for this principal.

R7. Users are allowed to join devices to Microsoft Entra ID

Description: The first user who signs in during OOBE must have **Entra join permission**. **Why:** The device Entra-joins under that user; if joining is blocked, provisioning fails at OOBE. (Microsoft, *Autopilot device preparation requirements → Configuration*.) **Verify:** `https://entra.microsoft.us` *(verify for GCC High)* → **Devices** → **Device settings** → **Users may join devices to Microsoft Entra** → set to **Selected** with the group `CFAPG-Provisioning-Users`, or **All**. **Implement:** Set the value and add pilot users to `CFAPG-Provisioning-Users` (created in `03-implementation.md`).

R8. Automatic (MDM) enrollment is enabled for Windows

Description: Windows **automatic enrollment** is turned on so an Entra-joined device auto-enrolls into Intune. **Why:** Device preparation relies on Entra join triggering Intune MDM enrollment. **Verify:** `https://intune.microsoft.us` → **Devices** → **Enrollment** → **Automatic Enrollment** (or **Windows enrollment**); confirm **MDM user scope** includes your provisioning users. **Implement:** Set **MDM user scope** to **Some** (`CFAPG-Provisioning-Users`) or **All**.

R9. Sovereign network endpoints allowed through filtering

Description: Devices and admins can reach the **US Gov** service endpoints outbound (HTTP 80, HTTPS 443, and UDP 123 for time), with **no SSL inspection** on the Intune/MDM endpoints. **Why:** Enrollment, policy sync, app delivery, activation, and diagnostics all depend on these URLs; SSL inspection on `*.manage.microsoft.us` breaks management. (Microsoft, *Network endpoints for US government deployments*.) **Why (allow-list, minimum):**

- `*.manage.microsoft.us` (unauthenticated proxy access required; **no SSL inspection**)
- `enterpriseregistration.microsoftonline.us`
- `login.microsoftonline.us`
- `graph.microsoft.us`
- `portal.manage.microsoft.us`
- `login.live.com` (Autopilot deployment service contact)
- `time.windows.com` (UDP 123, NTP)
- `*.msftconnecttest.com` (NCSI), `*.events.data.microsoft.com` (diagnostics), `lgmsapeweu.blob.core.windows.net` (device-prep diagnostics upload)
- Office 365 **US Gov** URL/IP ranges (via the Office 365 IP/URL service, worldwide feed includes Gov)

Verify: From a target network, resolve and reach `https://login.microsoftonline.us` and `https://portal.manage.microsoft.us`. `CFAPG-Discovery.ps1` records the endpoints in scope. **Implement:** Allow the URLs above through proxy/firewall; **exempt** `*.manage.microsoft.us` and Defender endpoints from SSL/TLS inspection.

R10. Understand the GCC High servicing model (no Autopatch / update policies)

Description: Windows servicing is delivered via **Update rings for Windows 10 and later** — **not** Windows Autopatch, feature/quality/expedite/driver update policies (all **not available** in GCC High). **Why:** Building the rollout around Autopatch or the newer update policies will fail because those blades/policies do not function in GCC High. (Microsoft, *Intune Government Service overview → features not available / planned*.) **Verify:** `https://intune.microsoft.us` → **Devices** → **Windows** → **Update rings**; confirm Update rings are available (they are). Note that **Feature updates**/**Quality updates** policy types are absent or non-functional. **Implement:** Plan `CFAPG-UpdateRing-Pilot` as the servicing mechanism (built in `03-implementation.md`).

Assess gate (pass/fail)

Do **not** proceed to Configure until **all** are TRUE:

- R1 — Tenant confirmed **GCC High or DoD** (not Commercial, not GCC).
- R2 — Qualifying **Gov** license with Intune + Entra ID P1/P2 assigned to pilot users.
- R3 — Operator has **Intune Administrator**; a Global Admin is available for setup.
- R4 — Target devices run **supported Windows 11** (24H2+, or 22H2/23H2 + KB5035942).
- R5 — Deployment planned as **Entra join** (not hybrid).
- R6 — **Intune Provisioning Client** (`f1346770-…`) present and can own a group.
- R7 — Users may **join devices to Entra** (`CFAPG-Provisioning-Users` or All).
- R8 — **Automatic enrollment** enabled for the provisioning users.
- R9 — Sovereign **endpoints allowed**, **no SSL inspection** on `*.manage.microsoft.us`.
- R10 — Servicing planned via **Update rings** (no Autopatch/update-policy dependency).

Any FALSE → fix before configuring. `CFAPG-Validation.ps1` automates R6–R8 checks against US Gov Graph.

Sources

- Microsoft, *Microsoft Intune Government Service overview.*
- Microsoft, *Overview of Windows Autopilot device preparation.*
- Microsoft, *Windows Autopilot device preparation requirements.*
- Microsoft, *Network endpoints for US government deployments.*
- Microsoft, *Windows Autopilot device preparation — create an assigned device group.*

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Implementation Guide — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

This guide builds the solution **step by step in the GCC High portals**. It assumes the **Assess** gate in `02-requirements.md` passed. Object names are used **verbatim** from the overview — reuse them exactly. GCC-High-only caveats appear **inline at the step where they matter**, boxed like this:

> ***GCC High caveat:*** *…*

Portals used (bookmark these):

- Intune admin center: `https://intune.microsoft.us`
- Entra admin center: `https://entra.microsoft.us` *(verify for GCC High)*
- Azure portal: `https://portal.azure.us`
- Microsoft 365 admin: `https://portal.office365.us`
- Sign-in: `https://login.microsoftonline.us`

> ***GCC High caveat (applies to the whole guide):*** *Every navigation path below is written for the `*.us` portals. If a screen or blade shown in a Commercial tutorial is missing here, it is almost always because the feature* ***is not available in GCC High*** *(see the parity table in `01-solution-overview.md` and `gcc-high-parity-notes.md`) — it is not a bug.* ***Do not*** *look for a classic "Windows Autopilot" / "Deployment Profiles" / "Devices import (hardware hash)" blade: classic Autopilot does not exist in GCC High.*

Phase A — Configure (build the objects)

Step 1 — Create the Microsoft Entra security groups

Sign in to `https://entra.microsoft.us` *(verify for GCC High)* as Intune Administrator.

1. **Groups** → **All groups** → **New group.**
2. Create `CFAPG-Provisioning-Users`

- **Group type:** Security
- **Membership type:** Assigned
- Add the pilot users who will provision devices. Select **Create**.

1. **New group** again → create `CFAPG-Pilot`

- **Group type:** Security · **Membership type:** Assigned

• Add the same pilot users (this group targets policies/apps to pilot). **Create**.

1. **New group** again → create `CFAPG-Autopilot-Devices`

• **Group type:** Security

• **Membership type: Assigned** — **not** Dynamic.

> ***GCC High caveat:*** *The device group **must** be **Assigned**. Enrollment Time Grouping (how devices land in the group during OOBE) works by the provisioning service principal *owning* an assigned group; a dynamic group breaks it.*

• Do **not** add members. Select **Create**.

Step 2 — Add the Intune Provisioning Client as owner of the device group

This is the step most often missed, and it is the reason nothing deploys during OOBE.

1. Still in `https://entra.microsoft.us` → **Groups** → **All groups** →

`CFAPG-Autopilot-Devices` → **Owners** → **Add owners**.

1. Search for **Intune Provisioning Client**.

> ***GCC High caveat:*** *In some tenants this principal is named **Intune Autopilot ConfidentialClient**. Match by **AppId** `f1346770-5b25-470b-88bd-d5744ab7952c`, not by name.*

1. Select it and **Add**. Confirm it now appears under **Owners**.

Run `CFAPG-Validation.ps1` later to confirm the ownership programmatically against US Gov Graph.

Step 3 — Allow users to join devices to Entra + enable automatic enrollment

1. `https://entra.microsoft.us` *(verify for GCC High)* → **Devices** → **Device settings**

→ **Users may join devices to Microsoft Entra** → set **Selected** and add `CFAPG-Provisioning-Users` (or **All**). **Save.**

1. `https://intune.microsoft.us` → **Devices** → **Enrollment** → **Automatic Enrollment**

→ set **MDM user scope** to **Some** and select `CFAPG-Provisioning-Users` (or **All**). Leave MDM terms/discovery URLs at default. **Save.**

> ***GCC High caveat:*** *The enrollment/discovery URLs default to the sovereign `*.manage.microsoft.us` service — do not paste Commercial `*.manage.microsoft.com` values.*

Step 4 — Create the least-privilege RBAC role `CFAPG Device Preparation Administrator`

1. `https://intune.microsoft.us` → **Tenant administration** → **Roles** → **All roles**

→ **Create** → **Intune role**.

1. **Basics:** Name = `CFAPG Device Preparation Administrator`; add a description.

2. **Permissions** — set exactly (all others **No**):

• **Device configurations:** Read = Yes, Create = Yes, Update = Yes, Delete = Yes,

Assign = Yes

• **Enrollment programs: Enrollment time device membership assignment = Yes**

• **Managed apps:** Read = Yes

• **Mobile apps:** Read = Yes

• **Organization:** Read = Yes

1. **Scope tags:** add `CFAPG` (create it if prompted). **Next** → **Create.**

2. **Assignments** → **Assign** → members = your admin group; scope (groups) = `CFAPG-Pilot`; scope tags = `CFAPG`.

(Permission set: Microsoft, *Autopilot device preparation requirements → RBAC*.)

Step 5 — Create the Enrollment Status Page (ESP) profile

1. `https://intune.microsoft.us` → **Devices** → **Enrollment** → **Enrollment Status Page** → **Create**.

1. Name = `CFAPG-ESP-Provisioning`.

2. **Settings:**

• **Show app and profile configuration progress** = **Yes**

• **Block device use until all apps and profiles are installed** = **Yes**

• Set a **timeout** (e.g. 60 minutes) and allow reset/retry.

• **Only show page to devices provisioned by out-of-box experience (OOBE)** = **Yes**

• Optionally require selected apps to install before desktop access (keep the list **short**).

> ***GCC High caveat:*** *Keep the OOBE-blocking app count small. Microsoft explicitly limits how many apps run during OOBE in device preparation "to increase stability and achieve a higher success rate." Heavy app lists reduce success rates.*

1. **Assignments:** assign to `CFAPG-Pilot`. **Create.**

Step 6 — Create a Windows update ring (the GCC High servicing mechanism)

> ***GCC High caveat:*** *Do **not** create Feature/Quality/Expedite/Driver update policies or plan Windows Autopatch — **none are available in GCC High.** Use **Update rings**.*

1. `https://intune.microsoft.us` → **Devices → Windows → Update rings for Windows 10 and later → Create profile**.

1. Name = `CFAPG-UpdateRing-Pilot`. Configure a conservative pilot ring (e.g. quality deferral 0–7 days, feature deferral per your policy, active hours).

1. **Assignments:** assign to `CFAPG-Pilot`. **Create.**

Step 7 — Create a compliance policy

1. `https://intune.microsoft.us` → **Devices** → **Compliance** → **Create policy** → Platform = **Windows 10 and later**.

1. Name = `CFAPG-Compliance-Win11`. Set baseline rules (e.g. BitLocker required, Secure Boot, minimum OS version matching R4, firewall on).

> ***GCC High caveat:*** *Do **not** rely on a compliance rule that requires **Device Health Attestation** results — DHA is **not available in GCC High**. Use settings that evaluate locally (BitLocker, Secure Boot, OS build, Defender status).*

1. **Assignments:** assign to `CFAPG-Pilot`. **Create.**

Step 8 — (Optional) Windows Hello for Business policy

1. `https://intune.microsoft.us` → **Devices → Enrollment → Windows → Windows Hello for Business**, or a device-configuration WHfB profile targeted to `CFAPG-Pilot`.

1. Configure PIN/biometric per your standard.

> ***GCC High caveat:*** *WHfB provisioning works, but the **Microsoft PIN reset service is not available in Azure Government**. Plan PIN recovery via allowed sign-in URLs (Intune/CSP/GPO) rather than the hosted PIN-reset service. *(Verify for GCC High for your exact build.)**

Step 9 — Add the applications/scripts to deploy during OOBE

1. `https://intune.microsoft.us` → **Apps → Windows → Add** — add your Win32 / line-of-business apps and **assign** them to `CFAPG-Autopilot-Devices`.

1. **Devices → Scripts and remediations** (or **Devices → Windows → PowerShell scripts**) — add any provisioning PowerShell scripts and assign to `CFAPG-Autopilot-Devices`.

> ***GCC High caveat:*** *Do **not** plan **Microsoft Store for Business** app provisioning (not available in GCC High) and do **not** rely on **Delivery Optimization for Win32 apps** peer caching (not available). Apps download from the cloud per device.*

Only the apps/scripts you also select **inside the device preparation profile** (next step) run during OOBE; anything else assigned to the group installs **after** provisioning completes.

Step 10 — Create the Autopilot device preparation profile

> ***This replaces classic Autopilot deployment profiles**, which do not exist in GCC High.*

1. `https://intune.microsoft.us` → **Devices → Windows → Enrollment** → tile **Windows Autopilot device preparation** → **Device preparation policies → Create**.

(If you do not see this tile, re-confirm R1 — you may be on a Commercial/GCC tenant.)

1. **Basics:** Name = `CFAPG-DevicePrep-UserDriven-EntraJoin`. **Next.**
2. **Configuration settings:**

• **Deployment settings → Deployment mode: User-driven**

> ***GCC High caveat:*** *Choose **User-driven**. **Self-deploying** and **pre-provisioning** modes are **not yet supported in GCC High** (planned).*

• **Join type: Microsoft Entra join** (the only supported type).

• **Device group:** select `CFAPG-Autopilot-Devices` (the assigned group owned by the

Intune Provisioning Client from Step 2).

• **Deployment settings** → **Allowed apps / scripts:** select the specific apps and

PowerShell scripts (from Step 9) to run **during OOBE**. Keep the list short.

• Leave **device rename during OOBE** unset.

> ***GCC High caveat:*** *Renaming the device during OOBE based on org structure is **not yet supported in GCC High** (planned). Rename post-provisioning if needed.*

1. **Assignments:** assign the profile to `CFAPG-Provisioning-Users` (device

preparation profiles target the *users* who provision). **Next** → **Create.**

Step 11 — (Optional) Corporate identifiers, only if blocking personal devices

If you use **enrollment restrictions** to block personal Windows enrollment, pre-upload trusted devices so they can still provision:

1. `https://intune.microsoft.us` → **Devices → Enrollment → Corporate device

identifiers → Add** → upload manufacturer/model/serial for the trusted devices.

> ***GCC High caveat:*** *This is **not** the classic Autopilot hardware-hash import (which does not exist here). Corporate identifiers are only needed when enrollment restrictions would otherwise block the device.*

Phase B — Pilot (provision real devices)

1. Confirm a pilot user is in `CFAPG-Provisioning-Users` and `CFAPG-Pilot`, is

Gov-licensed (R2), and may join devices (R7).

1. On a supported Windows 11 device (R4), **factory-fresh or reset**, begin OOBE:

• Connect to a network that reaches the sovereign endpoints (R9).

• At sign-in, enter the pilot user's **GCC High** account and authenticate at

`login.microsoftonline.us`.

1. The device Entra-joins, MDM-enrolls, is added to `CFAPG-Autopilot-Devices` at enrollment

time, shows the `CFAPG-ESP-Provisioning` progress percentage, installs the selected apps/scripts, applies `CFAPG-Compliance-Win11` and `CFAPG-UpdateRing-Pilot`, and lands on the desktop.

> ***GCC High caveat:*** *If OOBE finishes but the group is empty and nothing installed, the Intune Provisioning Client owner (Step 2) is almost certainly missing or wrong — see `06-troubleshooting.md`.*

Proceed to `05-testing.md` for the **Validate** gate, then `CFAPG-Set-ScopeTargeting.ps1` (default `-WhatIf`) to **Expand**.

Sources

• Microsoft, *Overview / requirements of Windows Autopilot device preparation* (Entra-join

only, user-driven only in GCC High, assigned device group, Intune Provisioning Client owner, RBAC permission set, OOBE app limit).

- Microsoft, *Intune Government Service overview* (features not available/planned in GCC

High: classic Autopilot, Autopatch/update policies, DFCI, DHA, Store for Business, self-deploying/pre-provisioning, OOBE rename).

- Microsoft, *Network endpoints for US government deployments* (portals, `*.us` endpoints).

Testing & Validation — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

This is the **Validate** gate. Every test has an explicit **pass/fail** condition. Do not **Expand** to more groups until all gates pass. Read-only scripts (`CFAPG-Validation.ps1`, `CFAPG-Discovery.ps1`, `CFAPG-Report.ps1`) automate much of this against **US Gov Graph** (`-Environment USGov`).

> ***GCC High reminder:*** *All portal checks use the* `*.us` *admin centers (*`https://intune.microsoft.us`*,* `https://entra.microsoft.us` **(verify for GCC High)*).*

T1 — Configuration objects exist and are correct (pre-pilot)

Run: `CFAPG-Validation.ps1` (read-only).

Check	Pass
Groups `CFAPG-Provisioning-Users`, `CFAPG-Pilot`, `CFAPG-Autopilot-Devices` exist	All three present
`CFAPG-Autopilot-Devices` is **Assigned** (not Dynamic)	membershipType = Assigned
Intune Provisioning Client (`f1346770-…`) is an **owner** of `CFAPG-Autopilot-Devices`	Owner present
Device preparation profile `CFAPG-DevicePrep-UserDriven-EntraJoin` exists, **User-driven**, **Entra join**	Confirmed
ESP `CFAPG-ESP-Provisioning` assigned to `CFAPG-Pilot`	Assigned
Update ring `CFAPG-UpdateRing-Pilot` + compliance `CFAPG-Compliance-Win11` assigned	Assigned

Fail → fix the missing/incorrect object in `03-implementation.md` before piloting. The owner check is the highest-value gate.

T2 — Tenant is genuinely GCC High (not Commercial)

Check: Signed into `https://intune.microsoft.us` you can see the **Windows Autopilot device preparation** tile, and you do **not** see a classic "Windows Autopilot" / "Deployment Profiles" import. `CFAPG-Discovery.ps1` reports the connected Graph environment as **USGov**. **Pass:** Graph environment = USGov; device-preparation tile present. **Fail:** If you see classic Autopilot blades or Graph reports Global/Commercial, you are on the wrong tenant/cloud — stop and re-confirm R1.

T3 — Endpoint reachability from the provisioning network

Check: From the network a pilot device will use, resolve and reach: `login.microsoftonline.us`, `portal.manage.microsoft.us`, `graph.microsoft.us`, `enterpriseregistration.microsoftonline.us`, `login.live.com`, and NTP `time.windows.com` (UDP 123). Confirm **no SSL inspection** on `*.manage.microsoft.us`. **Pass:** All resolve and connect; `*.manage.microsoft.us` is inspection-exempt. **Fail:** Any blocked/inspected → fix per R9 before piloting.

T4 — End-to-end pilot provisioning (the core test)

Run: provision one factory-fresh/reset supported Windows 11 device with a pilot user (per `03-implementation.md` Phase B).

Gate	Pass
Device Entra-joins during OOBE	Joined (visible later in Entra → Devices)
Device auto-enrolls into Intune	Appears in Intune → Devices → All devices
Device is added to `CFAPG-Autopilot-Devices` at enrollment	Member appears automatically
ESP `CFAPG-ESP-Provisioning` shows progress percentage	Percentage UI shown
Selected OOBE apps/scripts install	Status = installed/succeeded
User reaches desktop without technician touch	Zero-touch achieved
Device shows **compliant** against `CFAPG-Compliance-Win11`	Compliant (allow grace period)

Fail on empty group / no installs → Intune Provisioning Client owner missing (`06-troubleshooting.md`, symptom #1).

T5 — Deployment monitoring reflects the run

Check: `https://intune.microsoft.us` → **Devices** → **Windows** → **Windows Autopilot device preparation** → **Monitor** (deployment report). The pilot device appears with profile name, deployment status, per-app and per-script status. `CFAPG-Report.ps1` produces the same summary from US Gov Graph. **Pass:** The device and its app/script statuses are visible. **Fail:** No record → device likely never matched the profile (user not in `CFAPG-Provisioning-Users`, or device pre-registered as classic Autopilot — which takes precedence).

T6 — Servicing and policy land (post-provisioning)

Check: After provisioning, `CFAPG-UpdateRing-Pilot` and any post-OOBE apps assigned to `CFAPG-Autopilot-Devices` apply within a normal sync window. **Pass:** Update ring shows assigned/applied; post-OOBE apps install. **Fail:** Confirm assignments target the device group and the device is a member.

> ***GCC High reminder:*** *Do not test for Autopatch/feature-update-policy behavior — those are* ***not available in GCC High****. Servicing is validated only via the* ***update ring****.*

Validate gate (pass/fail summary)

Proceed to **Expand** only when **all** are TRUE:

- T1 — All objects correct; **Intune Provisioning Client owns the device group.**
- T2 — Confirmed **GCC High** (Graph USGov; device-preparation tile present).
- T3 — Sovereign endpoints reachable; **no SSL inspection** on `*.manage.microsoft.us`.
- T4 — One device provisioned **zero-touch** end to end and is compliant.
- T5 — Deployment **monitoring** shows the device with app/script status.
- T6 — Update ring + post-OOBE policy/apps applied.

Then run `CFAPG-Set-ScopeTargeting.ps1` (default `-WhatIf`) to preview widening scope, review, and re-run with `-WhatIf:$false` to apply.

Sources

- Microsoft, *Windows Autopilot device preparation — overview, requirements, reporting and monitoring.*
- Microsoft, *Intune Government Service overview* (GCC High feature availability).
- Microsoft, *Network endpoints for US government deployments.*

Troubleshooting — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

Symptoms and fixes, ordered by how often they bite on a **GCC High / DoD** tenant. Many are GCC-High-specific (wrong cloud, missing sovereign feature, wrong portal). Run `CFAPG-Validation.ps1` and `CFAPG-Discovery.ps1` (read-only, `-Environment USGov`) to gather evidence before changing anything.

> ***Portal reminder:*** *troubleshoot in the* `*.us` *portals —* `https://intune.microsoft.us`*,* `https://entra.microsoft.us` *(verify for GCC High)*,* `https://portal.azure.us`*.*

1. OOBE completes but the device group is empty and nothing installed

By far the most common failure. The device Entra-joins and enrolls, but no apps/scripts run and `CFAPG-Autopilot-Devices` has no members.

Cause: The **Intune Provisioning Client** service principal (`f1346770-5b25-470b-88bd-d5744ab7952c`) is **not an owner** of the assigned device group, so **Enrollment Time Grouping** can't add the device. **Fix:**

1. `CFAPG-Validation.ps1` → confirm the "Provisioning Client OWNS …" check is **FAIL**.
2. `CFAPG-Remediation.ps1` (preview), then `-WhatIf:$false` to add the owner — **or**

`entra.microsoft.us` → **Groups** → `CFAPG-Autopilot-Devices` → **Owners** → **Add owners** → add the principal (match by **AppId**, may be named *Intune Autopilot ConfidentialClient*).

1. Reset the device and re-provision.

2. The device group is Dynamic, not Assigned

Cause: `CFAPG-Autopilot-Devices` was created as a **Dynamic** group; Enrollment Time Grouping requires an **Assigned** group and the owner mechanism does not apply. **Fix:** Recreate the group as **Assigned** (`03-implementation.md` Step 1), re-add the Provisioning Client owner (Step 2), point the device preparation profile at it, re-provision.

3. "I don't see Windows Autopilot / Deployment Profiles / hardware-hash import"

Cause: This is **expected** — **classic Windows Autopilot is not available in GCC High/DoD** (and there is no plan to add it). You are looking for a Commercial-only surface. **Fix:** Use **Windows Autopilot device preparation** instead (`intune.microsoft.us` → **Devices** → **Windows** → **Enrollment** → **Windows Autopilot device preparation**). Do not import hardware hashes; the user-driven flow does not need them. (*Confirmed: Microsoft, Intune Government Service overview.*)

4. Scripts return nothing / see the "wrong" tenant

Cause: The Graph session connected to the **Commercial** cloud, not US Gov. **Fix:** Always pass `-Environment USGov` (or `USGovDoD`). Every script prints `Environment = …` on connect — confirm it says **USGov**. If it says Global, disconnect (`Disconnect-MgGraph`) and reconnect with the correct environment. The scripts refuse to make changes when the environment isn't US Gov.

5. Provisioning fails at sign-in / can't reach services

Cause: Sovereign endpoints blocked, or **SSL inspection** on `*.manage.microsoft.us`. **Fix:** Allow (and exempt from SSL inspection) `*.manage.microsoft.us`, `login.microsoftonline.us`, `enterpriseregistration.microsoftonline.us`, `graph.microsoft.us`, `login.live.com`, `*.msftconnecttest.com`, `*.events.data.microsoft.com`, `lgmsapeweu.blob.core.windows.net`, and NTP UDP 123 to `time.windows.com` (R9). Do **not** paste Commercial `*.manage.microsoft.com` values.

6. Device won't provision — unsupported Windows build

Cause: Device runs **Windows 10** or a Windows 11 build without **KB5035942**. **Fix:** Device preparation requires **Windows 11 24H2+** or **22H2/23H2 with KB5035942** (April 2024 media or later). Re-image/refresh with compliant media (R4). Windows 10 is not supported for device preparation.

7. Self-deploying / pre-provisioning / device-rename options are missing or ignored

Cause: These modes are **not yet available in GCC High** (Microsoft lists them as "planned"). **Fix:** Use **user-driven** deployment; rename devices **after** provisioning. Do not design kiosk/self-deploying flows on GCC High yet. (*Confirmed: Intune Government Service overview.*)

8. Too many apps during OOBE / low provisioning success rate

Cause: Large OOBE app list — Microsoft deliberately limits apps during device-prep OOBE for stability. **Fix:** In `CFAPG-DevicePrep-UserDriven-EntraJoin` select only the **essential** apps/ scripts for OOBE; assign the rest to `CFAPG-Autopilot-Devices` for post-provisioning install. Keep the ESP blocking app list short.

9. Windows Hello for Business PIN reset doesn't work

Cause: The **Microsoft hosted PIN reset service is not available in Azure Government. Fix:** Configure PIN recovery via allowed sign-in URLs through Intune/CSP/GPO rather than the hosted service. Validate on a pilot device. *(Verify for GCC High for your exact Windows build.)*

10. Update/servicing policies don't appear or don't apply

Cause: You configured **Windows Autopatch** or **Feature/Quality/Expedite/Driver update policies** — **none are available in GCC High. Fix:** Service via **Update rings for Windows 10 and later** (`CFAPG-UpdateRing-Pilot`). Remove any dependency on Autopatch or update-policy reports.

11. Compliance policy always evaluates non-compliant on an attestation rule

Cause: A rule requires **Device Health Attestation** results — **DHA is not available in GCC High. Fix:** Use locally evaluated rules (BitLocker, Secure Boot, OS build, Defender) in `CFAPG-Compliance-Win11`; remove DHA-dependent rules.

12. Certificate provisioning expected via Cloud PKI

Cause: Cloud PKI is not available in GCC High. Fix: Use **SCEP/PKCS** with your own PKI/NDES for certificate delivery instead.

13. Remote assistance for a stuck device isn't available

Cause: Remote Help is not available in GCC High (planned). **Fix:** Use your organization's approved alternative remote-assist tool for post-provisioning support (document it in your runbook).

14. Device was previously a classic Autopilot device elsewhere

Cause: If a device is registered/added as a **Windows Autopilot** device, that profile **takes precedence** over the device preparation policy. **Fix:** Deregister the device from Windows Autopilot, then re-provision via device preparation. (*Confirmed: Autopilot device preparation overview.*)

15. RBAC admin can't configure device preparation

Cause: The role lacks **Enrollment programs** → **Enrollment time device membership assignment** (or the Device configurations set). **Fix:** Confirm `CFAPG Device Preparation Administrator` has exactly the permissions in `03-implementation.md` Step 4, including **Enrollment time device membership assignment = Yes**.

Escalation checklist (attach to any support case)

- Output of `CFAPG-Discovery.ps1` and `CFAPG-Validation.ps1` (confirming **USGov**).
- The device group's **membership type** (Assigned) and **Owners** list.
- The device preparation profile mode (**user-driven**) and **join type** (**Entra join**).
- Confirmation the tenant is **GCC High/DoD** (not Commercial/GCC).
- Endpoint/proxy config showing sovereign URLs allowed and `*.manage.microsoft.us` exempt from SSL inspection.
- The Windows build on the failing device (`winver`).

Sources

- Microsoft, *Intune Government Service overview* (not-available/planned features).
- Microsoft, *Overview / requirements of Windows Autopilot device preparation* (Assigned group + Provisioning Client owner, Entra-join only, OOBE app limit, Autopilot precedence).
- Microsoft, *Network endpoints for US government deployments* (sovereign endpoints, no SSL inspection).
- Microsoft, *Configure Windows Hello for Business* (PIN reset service unavailable in Azure Government).

Best Practices — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

Practices tuned for the **US sovereign cloud**. Where a practice exists because of a GCC High parity gap, that is called out.

Portals & tooling

- **Bookmark the** `*.us` **portals** and remove Commercial bookmarks from the admin's browser

profile to prevent muscle-memory mistakes: `intune.microsoft.us`, `entra.microsoft.us` *(verify for GCC High)*, `portal.azure.us`, `portal.office365.us`, `login.microsoftonline.us`.

- **Always connect Graph with** `-Environment USGov`**.** A script that silently connects to

Commercial Graph will read an empty/foreign tenant. Every script in this solution does this and prints the environment on connect — verify it says **USGov**.

- **Treat "missing blade" as "not available in GCC High"** first, not as a bug. Cross-check

`gcc-high-parity-notes.md` before opening a support case.

Provisioning design (the supported path only)

- **Design on Autopilot *device preparation*, never classic Autopilot.** Classic Autopilot

(hardware-hash registration, deployment profiles, self-deploying kiosks) does not exist in GCC High. Do not carry a Commercial hardware-hash CSV process into this tenant.

- **Use an *Assigned* device security group** and make the **Intune Provisioning Client**

(`f1346770-5b25-470b-88bd-d5744ab7952c`) an **owner** of it. This is the single most common failure point — verify ownership after every group change with `CFAPG-Validation.ps1`.

- **Keep the OOBE app/script list short.** Microsoft deliberately limits apps during OOBE

in device preparation for stability. Deliver the minimum during provisioning; push the rest post-OOBE via assignment to `CFAPG-Autopilot-Devices`.

- **Entra join only.** Plan on-prem resource access (if needed) as a *post-provisioning*

add-on (cloud Kerberos trust / hybrid access), not as hybrid Autopilot join — which is unsupported by device preparation.

- **Do not plan self-deploying, pre-provisioning, or OOBE device-rename** — all "planned"

but not yet available in GCC High. Rename devices post-provisioning if required.

Servicing (no Autopatch in GCC High)

- **Service via Update rings for Windows 10 and later.** Autopatch and the newer

feature/quality/expedite/driver update policies are **not available** in GCC High. Standardize on rings (`CFAPG-UpdateRing-Pilot` and successors) with staged deferrals.

- **Do not build reporting or SLAs on feature-update reports** — those reports are not

available in GCC High.

- **Do not rely on Delivery Optimization Win32 peer caching** — not available; size network

egress for per-device cloud downloads.

Compliance & security

- **Write compliance rules that evaluate locally** (BitLocker, Secure Boot, OS build,

Defender). **Do not** require **Device Health Attestation** results — DHA is not available in GCC High.

• **Cloud PKI is not available in GCC High** — if you need certificate provisioning, plan SCEP/PKCS with your own PKI/NDES rather than Intune Cloud PKI.

• **Remote Help is not available** — pre-select an approved alternative remote-assist tool for post-provisioning support, and document it in your runbook.

• **WHfB PIN recovery:** the hosted Microsoft PIN reset service is unavailable in Azure Government. Configure PIN reset via allowed sign-in URLs (Intune/CSP) and test it *(verify for GCC High for your build)*.

RBAC & governance

• **Run day-to-day with the least-privilege role** `CFAPG Device Preparation Administrator` (only the permissions device preparation needs), not Global/Intune Admin.

• **Apply the** `CFAPG` **scope tag** to this solution's objects and role assignment so government segregation-of-duties is auditable.

• **Consider the GCC High STIG.** Microsoft points GCC High admins to the DISA STIG for MDM hardening — align your compliance and configuration baselines to it.

Operations & change control

• **Gate every expansion.** Keep the `Assess → Configure → Pilot → Validate → Expand` discipline; widen scope only with `CFAPG-Set-ScopeTargeting.ps1` after previewing with `-WhatIf`.

• **Re-validate after any Entra group edit** — a lost owner on the device group silently breaks all future provisioning.

• **Watch the docs for GCC High parity changes.** GCC High capability is expanding (device preparation itself is recent); re-check `gcc-high-parity-notes.md` items marked "planned" or "verify for GCC High" each quarter.

• **Never inspect SSL on** `*.manage.microsoft.us` or Defender endpoints — it breaks management and is unsupported.

Sources

• Microsoft, *Intune Government Service overview* (not-available / planned features).

• Microsoft, *Windows Autopilot device preparation — overview & requirements* (OOBE app limit, Entra-join only, assigned group + provisioning-client owner, RBAC).

• Microsoft, *Network endpoints for US government deployments* (no SSL inspection on `*.manage.microsoft.us`).

Deliverables Summary — Cloud-First Autopilot Zero-Touch Provisioning (Government / GCC High)

Everything included in this Government (GCC High) edition, and where to start.

Documents (`source/`)

Document	What it gives you
`executive-summary.md`	One-page situation/complication/resolution for decision-makers — why GCC High is not "Commercial with a different login."
`01-solution-overview.md`	The zero-touch model in GCC High, the **Commercial vs GCC High — what's different** section, the standard deployment model, and the named objects.
`02-requirements.md`	Every solution-specific prerequisite with **Description · Why · Verify · Implement**, and the **Assess** pass/fail gate.
`03-implementation.md`	Ordered build steps in the `*.us` **portals**, with GCC-High-only caveats **inline** at each step.
`05-testing.md`	The **Validate** gate — T1–T6 with explicit pass/fail conditions.
`06-troubleshooting.md`	GCC-High-specific failure modes and fixes (empty device group, wrong cloud, blocked endpoints, WHfB PIN reset, etc.).
`07-best-practices.md`	Sovereign-cloud practices, including the servicing-without-Autopatch model.
`gcc-high-parity-notes.md`	The **cited** Commercial-vs-GCC-High reference; doc-confirmed vs "verify for GCC High."
`global-requirements-intune-solutions.md`	Category-level prerequisites shared by all Intune solutions (provided; not modified here).

PowerShell scripts (`scripts/`)

All connect to **Microsoft Graph US Gov** with `-Environment USGov` (USGovDoD supported), print the resolved environment before acting, and use hyphenated placeholders (e.g. `your-tenant-id`) — never angle brackets.

Script	Type	Default safety	Purpose
`CFAPG-Discovery.ps1`	Read-only	n/a	Inventory the tenant's provisioning posture (groups, device-prep profiles, provisioning-client SP, enrollment configs).
`CFAPG-Validation.ps1`	Read-only	n/a	Pass/fail the build: groups exist, device group is **Assigned**, and the **Intune Provisioning Client owns** it.
`CFAPG-Report.ps1`	Read-only	n/a	Rollout report — Entra-join/enrollment/compliance state of devices in `CFAPG-Autopilot-Devices`, optional CSV.
`CFAPG-Remediation.ps1`	Changes state	`-WhatIf` **(preview) by default**	Adds the Intune Provisioning Client as **owner** of the device group (the #1 fix). Applies only with `-WhatIf:$false`.
`CFAPG-Set-ScopeTargeting.ps1`	Changes state	`-WhatIf` **(preview) by default**	Widen scope during **Expand** by adding members to `CFAPG-Provisioning-Users`. Applies only with `-WhatIf:$false`.

Prerequisite for scripts: Microsoft Graph PowerShell SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`). Sign in with an account that holds the read (or, for the two write scripts, write) permissions listed in each script header.

Named objects created (verbatim)

`CFAPG-Provisioning-Users` · `CFAPG-Autopilot-Devices` (assigned; owner = Intune Provisioning Client) · `CFAPG-Pilot` · `CFAPG-DevicePrep-UserDriven-EntraJoin` · `CFAPG-ESP-Provisioning` · `CFAPG Device Preparation Administrator` (RBAC role) · `CFAPG-UpdateRing-Pilot` · `CFAPG-Compliance-Win11` · `CFAPG` (scope tag).

Suggested order of use

1. Read `executive-summary.md` and `01-solution-overview.md` (esp. **Commercial vs GCC High**).
1. Work the **Assess** gate in `02-requirements.md`; run `CFAPG-Discovery.ps1`.
2. Build per `03-implementation.md`; run `CFAPG-Validation.ps1` — fix any FAIL (use `CFAPG-Remediation.ps1` in preview first).

1. Pilot, then run the **Validate** gate in `05-testing.md` and `CFAPG-Report.ps1`.
2. **Expand** with `CFAPG-Set-ScopeTargeting.ps1` (preview, then apply).
3. Keep `06-troubleshooting.md`, `07-best-practices.md`, and `gcc-high-parity-notes.md`

on hand throughout.

The one thing to remember

Classic Windows Autopilot does not exist in GCC High. This solution delivers zero-touch provisioning entirely on the supported **Windows Autopilot device preparation** path — right portals, right technology, parity gaps handled.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CFAPG-Discovery.ps1

Purpose: Connects to Microsoft Graph US Gov and reports the current state relevant to Cloud-First Autopilot Zero-Touch Provisioning on a GCC High / DoD tenant: the tenant/environment, Autopilot device preparation profiles, the named CFAPG groups, and the Intune Provisioning Client service principal. Read-only. It changes nothing; it only describes what exists so you can plan the build. Placeholders use hyphenated names such as your-tenant-id and your-group-name. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CFAPG-Discovery.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, Application.Read.All EXPECTED RESULTS A printed summary. PASS when the connected environment prints as USGov and the report renders. FAIL (stop and re-check the tenant/cloud) if the environment is not USGov or connection fails. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\CFAPG-Discovery.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, DeviceManagementServiceConfig.Read.All, DeviceManagementConfiguration.Read.All, Application.Read.All EXPECTED RESULTS A printed summary. PASS when the connected environment prints as USGov and the report renders. FAIL (stop and re-check the tenant/cloud) if the environment is not USGov or connection fails. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

CFAPG-Remediation.ps1

Purpose: Connects to Microsoft Graph US Gov and remediates the top failure of Autopilot device preparation on a GCC High / DoD tenant: the Intune Provisioning Client service principal not being an OWNER of the assigned device group (which leaves the group empty during OOBE so nothing deploys). Runs in preview mode by default (-WhatIf) and only makes a change when you pass -WhatIf:$false. Placeholders use hyphenated names such as your-tenant-id and your-device-group-name. HOW TO RUN Preview (no change): powershell -ExecutionPolicy Bypass -File .\CFAPG-Remediation.ps1 Apply the fix: powershell -ExecutionPolicy Bypass -File .\CFAPG-Remediation.ps1 -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Application.Read.All, Directory.Read.All EXPECTED RESULTS Preview prints the intended action and makes no change. Apply adds the Provisioning Client as owner and confirms. PASS when ownership is present afterward; re-run CFAPG-Validation.ps1 to confirm. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the single, explicit owner-add performed only when -WhatIf:$false is passed. #>

How to run: Preview (no change): powershell -ExecutionPolicy Bypass -File .\CFAPG-Remediation.ps1 Apply the fix: powershell -ExecutionPolicy Bypass -File .\CFAPG-Remediation.ps1 -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Application.Read.All, Directory.Read.All EXPECTED RESULTS Preview prints the intended action and makes no change. Apply adds the Provisioning Client as owner and confirms. PASS when ownership is present afterward; re-run CFAPG-Validation.ps1 to confirm. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the single, explicit owner-add performed only when

-WhatIf:$false is passed. #>

CFAPG-Report.ps1

Purpose: Connects to Microsoft Graph US Gov and produces a human-readable and optional CSV report of provisioning outcomes on a GCC High / DoD tenant: devices in the CFAPG-Autopilot-Devices group, their Entra-join/enrollment/compliance state, and a rollout summary. Read-only. Use it after piloting and during expansion to evidence results. Placeholders use hyphenated names such as your-tenant-id and your-output-folder. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CFAPG-Report.ps1 (Graph scopes; -Environment USGov) Optional: -CsvPath your-output-folder\CFAPG-report.csv Scopes requested: Group.Read.All, Directory.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A printed rollout summary (counts by state) and per-device table; a CSV if -CsvPath is given. PASS when the report renders against the USGov environment. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\CFAPG-Report.ps1 (Graph scopes; -Environment USGov) Optional: -CsvPath your-output-folder\CFAPG-report.csv Scopes requested: Group.Read.All, Directory.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A printed rollout summary (counts by state) and per-device table; a CSV if -CsvPath is given. PASS when the report renders against the USGov environment. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

CFAPG-Set-ScopeTargeting.ps1

Purpose: Connects to Microsoft Graph US Gov and adds members (users or groups) to the CFAPG provisioning scope on a GCC High / DoD tenant during the Expand phase — for example adding a new business-unit group to CFAPG-Provisioning-Users so more users can provision devices. Runs in preview mode by default (-WhatIf) and only changes membership when you pass -WhatIf:$false. Placeholders use hyphenated names such as your-tenant-id, your-target-group-name, and your-member-object-id. HOW TO RUN Preview (no change): powershell -ExecutionPolicy Bypass -File .\CFAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id Apply: powershell -ExecutionPolicy Bypass -File .\CFAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Directory.Read.All EXPECTED RESULTS Preview lists the members that would be added and changes nothing. Apply adds them and confirms. PASS when the target group contains the intended members afterward. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the explicit membership add performed only when -WhatIf:$false is passed. #>

How to run: Preview (no change): powershell -ExecutionPolicy Bypass -File .\CFAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id Apply: powershell -ExecutionPolicy Bypass -File .\CFAPG-Set-ScopeTargeting.ps1 -AddMemberObjectIds your-member-object-id -WhatIf:$false (Graph scopes; -Environment USGov) Scopes requested: Group.ReadWrite.All, Directory.Read.All EXPECTED RESULTS Preview lists the members that would be added and changes nothing. Apply adds them and confirms. PASS when the target group contains the intended members afterward. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints except the explicit membership add performed only when -WhatIf:$false is passed. #>

CFAPG-Validation.ps1

Purpose: Connects to Microsoft Graph US Gov and validates the Cloud-First Autopilot Zero-Touch Provisioning configuration on a GCC High / DoD tenant against explicit pass/fail gates: the named groups exist, the device group is Assigned (not Dynamic), and the Intune Provisioning Client service principal OWNS the device group (the requirement that makes Enrollment Time Grouping work). Read-only. Placeholders use hyphenated names such as your-tenant-id. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CFAPG-Validation.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, Application.Read.All, DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall result. Overall PASS means the build is ready to pilot. Any FAIL prints the fix location in 03-implementation.md. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\CFAPG-Validation.ps1 (Graph scopes; -Environment USGov) Scopes requested: Directory.Read.All, Group.Read.All, Application.Read.All, DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall result. Overall PASS means the build is ready to pilot. Any FAIL prints the fix location in 03-implementation.md. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>